

Machine Learning Engineering

Taller 2: Ambientes virtuales de Python

Nombre: Mathew Cordero

Carné: 22982 **Fecha:** 29 de Sep

Repositorio: Repo

1. Descripción del ejercicio

En este taller se explora la creación y administración de ambientes virtuales con Python y Conda. También se definen los archivos de dependencias necesarios para reproducir un proyecto de Machine Learning.

2. Ambiente virtual de Python

2.1 Documentación consultada

- Documentación oficial de Python sobre `venv`: <https://docs.python.org/3/library/venv.html>
- Documentación de VS Code sobre ambientes de Python: <https://code.visualstudio.com/docs/python/environments>

2.2 Creación del ambiente

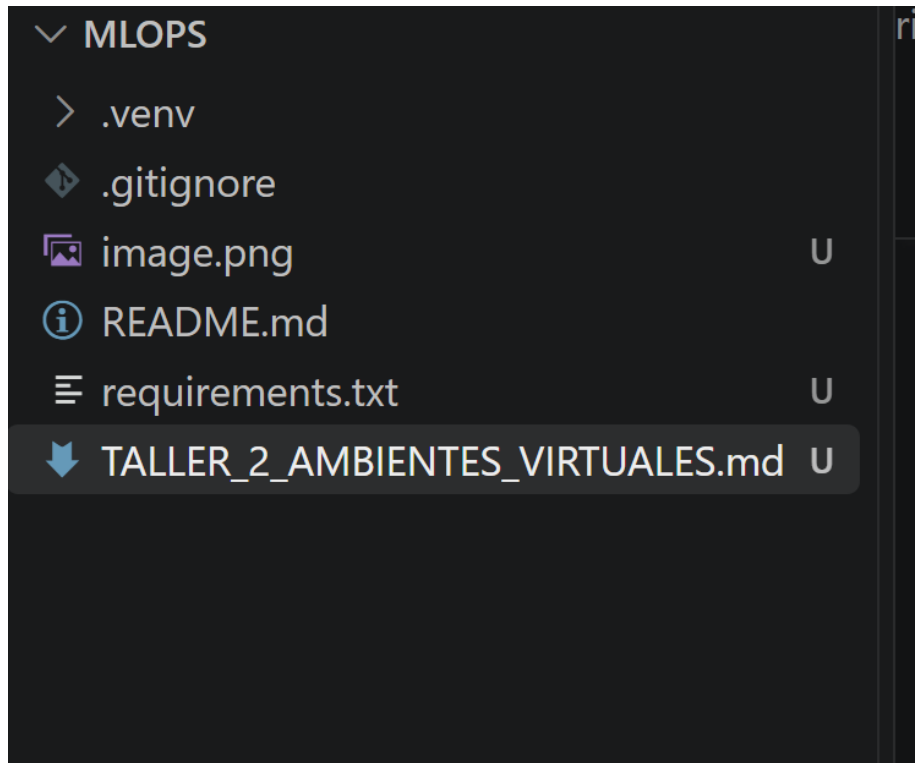
Comandos utilizados en PowerShell:

```
python --version
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

Para comprobar que el ambiente quedó activo:

```
python -c "import sys; print(sys.executable)"
pip --version
```

2.3 Evidencia

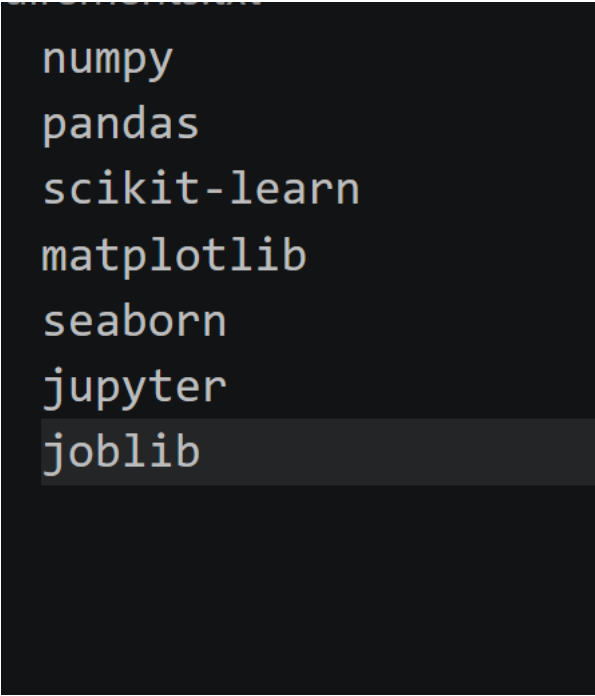


2.4 Archivo de requisitos

Por convención, el archivo de dependencias utilizado con `pip` se llama `requirements.txt`.

Contenido utilizado:

```
numpy
pandas
scikit-learn
matplotlib
seaborn
jupyter
joblib
```



```
numpy
pandas
scikit-learn
matplotlib
seaborn
jupyter
joblib
```

Instalación y verificación:

```
python -m pip install -r requirements.txt
python -m pip check
```

3. Ambiente virtual con Anaconda/Conda

3.1 Documentación consultada

- Administración de ambientes Conda: <https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html>

4.2 Creación del ambiente

Comandos utilizados en Anaconda Prompt o en una terminal con Conda disponible:

```
conda --version
conda create --name ml-taller2 python=3.12 -y
conda activate ml-taller2
python --version
conda env list
```

Instalación de paquetes:

```
conda install numpy pandas scikit-learn matplotlib seaborn jupyter joblib -y
```

4.3 Archivo de ambiente de Conda

Conda suele definir ambientes reproducibles mediante un archivo llamado `environment.yml`:

```
name: ml-taller2
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.12
  - numpy
  - pandas
  - scikit-learn
  - matplotlib
  - seaborn
  - jupyter
  - joblib
  - pip
```

El archivo puede generarse desde el ambiente actual:

```
conda env export --from-history > environment.yml
```

También puede usarse para recrear el ambiente:

```
conda env create -f environment.yml
```

4.4 Evidencia Conda

```
(base) PS C:\Users\cor22982\Documents\MLOPS> conda activate ml-taller2
(base) PS C:\Users\cor22982\Documents\MLOPS> conda activate ml-taller2
(ml-taller2) PS C:\Users\cor22982\Documents\MLOPS>
```

4. Comparación con otras herramientas

Para las siguientes tablas se toma como herramienta base el flujo tradicional descrito en este taller: `venv` para crear el ambiente virtual, `pip` para instalar paquetes y `requirements.txt` para registrar dependencias.

4.1 Comparación de venv/pip con UV

Indicación: esta tabla ya está redactada. El grupo debe revisarla con el artículo de DataCamp y adaptarla si desea incluir hallazgos propios.

Criterio	venv + pip
Propósito principal	<code>venv</code> crea ambientes aislados y <code>pip</code> instala paquetes.
Velocidad	La resolución e instalación puede ser más lenta en proyectos grandes.

Criterio	venv + pip
Creación del ambiente	Se ejecuta <code>python -m venv .venv</code> y luego se activa manualmente.
Instalación de paquetes	Se utiliza <code>python -m pip install paquete</code> .
Archivos principales	Normalmente utiliza <code>requirements.txt</code> ; el bloqueo exacto requiere prácticas o l
Reproducibilidad	Depende de fijar versiones correctamente, por ejemplo mediante <code>pip freeze</code> .
Administración de Python	Usa una instalación de Python que ya debe estar disponible en el sistema.
Compatibilidad	Es el flujo estándar incluido con Python y tiene amplia compatibilidad.
Curva de aprendizaje	Es sencillo y apropiado para aprender los fundamentos.
Uso recomendado	Scripts sencillos, talleres introductorios y proyectos con pocas dependencias.

Ejemplo de un flujo básico con UV:

```
uv init
uv add numpy pandas scikit-learn joblib
uv run python --version
```

4.2 Comparación de venv/pip con Poetry

Indicación: esta tabla ya está redactada. El grupo debe revisarla con el artículo de DataCamp y adaptarla si desea incluir hallazgos propios.

Criterio	venv + pip
Propósito principal	Combina una herramienta de ambientes con un instalador de paquetes.
Creación del ambiente	Se crea y activa manualmente con <code>python -m venv .venv</code> .
Instalación de paquetes	Se utiliza <code>python -m pip install paquete</code> y se actualiza manualmente requi
Archivos principales	Generalmente <code>requirements.txt</code> ; la metadata del paquete puede estar en <code>pyproject.toml</code> .
Resolución de dependencias	<code>pip</code> instala dependencias compatibles, pero el flujo tradicional no ofrece un arch
Reproducibilidad	Requiere versiones fijadas o un archivo generado con <code>pip freeze</code> .
Grupos de dependencias	Normalmente requiere varios archivos, por ejemplo <code>requirements-dev.txt</code> .
Construcción y publicación	Requiere herramientas adicionales, como <code>build</code> y <code>twine</code> .
Ejecución en el ambiente	Se activa el ambiente antes de ejecutar el programa.
Curva de aprendizaje	Tiene pocos conceptos iniciales y comandos conocidos.
Uso recomendado	Proyectos pequeños, aprendizaje y compatibilidad con flujos existentes.

Ejemplo de un flujo básico con Poetry:

```
poetry init
poetry add numpy pandas scikit-learn joblib
poetry run python --version
```

4.3 Análisis

Las tres opciones aíslan dependencias, pero su alcance es diferente. `venv` y `pip` ofrecen un flujo básico, transparente y ampliamente compatible. UV busca

cubrir el ciclo de administración de dependencias con mayor velocidad y una experiencia unificada. Poetry añade resolución, bloqueo, organización del proyecto y publicación de paquetes mediante un flujo más estructurado. Para un pipeline de Machine Learning compartido por un equipo, UV o Poetry pueden mejorar la reproducibilidad; para aprender cómo funciona el aislamiento de Python o mantener un proyecto sencillo, `venv` y `pip` siguen siendo suficientes.

5. Conclusiones

1. Los ambientes virtuales permiten aislar las dependencias de cada proyecto y evitan conflictos entre versiones de paquetes.
2. Un archivo como `requirements.txt` o `environment.yml` facilita que otras personas puedan reproducir el entorno de desarrollo y ejecutar el pipeline.
3. En proyectos de Machine Learning, fijar las versiones ayuda a obtener resultados consistentes durante el entrenamiento, las pruebas y el despliegue.
4. `venv` es una alternativa ligera incluida con Python, mientras que Conda también puede administrar la versión de Python y dependencias que no pertenecen exclusivamente al ecosistema de `pip`.
5. Mantener ambientes separados para desarrollo, pruebas y producción reduce errores y mejora la trazabilidad del proyecto.

9. Referencias

- Python Software Foundation. *Creation of virtual environments*. <https://docs.python.org/3/library/venv.html>
- Microsoft. *Python environments in VS Code*. <https://code.visualstudio.com/docs/python/environments>
- Conda. *Managing environments*. <https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html>
- pip. *Requirements File Format*. <https://pip.pypa.io/en/stable/reference/requirements-file-format/>
- DataCamp. *Python UV: The Ultimate Guide to the Fastest Python Package Manager*. <https://www.datacamp.com/tutorial/python-uv>
- DataCamp. *Python Poetry: Modern and Efficient Python Environment and Dependency Management*. <https://www.datacamp.com/tutorial/python-poetry>

10. Lista de verificación antes de entregar

- ☐ Completamos nombres, carnés y fecha.
- ☐ Pegamos el enlace del repositorio.
- ☐ Incluimos `requirements.txt`.
- ☐ Incluimos `environment.yml`.
- ☐ Pegamos la captura del ambiente creado con `venv`.
- ☐ Pegamos la captura del ambiente creado con Conda.
- ☐ Documentamos los requisitos del pipeline de scikit-learn.

- ☐ Revisamos las tablas comparativas con UV y Poetry.
- ☐ Escribimos los nombres de todos los integrantes de Project Groups.
- ☐ Redactamos o adaptamos las conclusiones.
- ☐ Verificamos que las imágenes se visualicen en Markdown.